

## SKILL WEEK – 9

### Session – 17

**TASK 1:** To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
keerthi_priya@LAPTOP-OSGV × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX_JOBS 10

struct Job {
    int job_id;
    pid_t pid;
    char command[50];
    char status[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command) {
    if (job_count < MAX_JOBS) {
        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;

        snprintf(jobs[job_count].command,
                 sizeof(jobs[job_count].command), "%s", command);

        snprintf(jobs[job_count].status,
                 sizeof(jobs[job_count].status), "Running");

        job_count++;
    }
}

void update_jobs() {
    for (int i = 0; i < job_count; i++) {
        int result = waitpid(jobs[i].pid, NULL, WNOHANG);

        if (result == jobs[i].pid) {
            snprintf(jobs[i].status,
                     sizeof(jobs[i].status), "Completed");
        }
    }
}

void list_jobs() {
    update_jobs();
}
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
void list_jobs() {
    update_jobs();

    printf("\nActive Jobs:\n");
    printf("JobID\tPID\tStatus\t\tCommand\n");
    printf("-----\n");

    for (int i = 0; i < job_count; i++) {
        printf("[%d]\t%d\t%-10s\t%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].status,
            jobs[i].command);
    }
}

int main() {
    pid_t pid1, pid2;

    printf("Job Management System\n");

    pid1 = fork();

    if (pid1 == 0) {
        sleep(3);
        exit(0);
    }

    if (pid1 > 0) {
        add_job(pid1, "sleep 3");
    }

    pid2 = fork();

    if (pid2 == 0) {
        sleep(5);
        exit(0);
    }

    if (pid2 > 0) {
        add_job(pid2, "sleep 5");
    }

    sleep(1);
}
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
        jobs[i].job_id,
        jobs[i].pid,
        jobs[i].status,
        jobs[i].command);
    }
}

int main() {
    pid_t pid1, pid2;

    printf("Job Management System\n");

    pid1 = fork();

    if (pid1 == 0) {
        sleep(3);
        exit(0);
    }

    if (pid1 > 0) {
        add_job(pid1, "sleep 3");
    }

    pid2 = fork();

    if (pid2 == 0) {
        sleep(5);
        exit(0);
    }

    if (pid2 > 0) {
        add_job(pid2, "sleep 5");
    }

    sleep(1);

    list_jobs();

    printf("\nWaiting for jobs to complete...\n");
    sleep(4);

    list_jobs();

    return 0;
}
```

```
keerthi_priya@LAPTOP-OSGV X + v
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session17_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session17_task1.c -o session17_task1
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session17_task1
Job Management System

Active Jobs:
JobID  PID      Status      Command
-----
[1]    520     Running     sleep 3
[2]    521     Running     sleep 5

Waiting for jobs to complete...

Active Jobs:
JobID  PID      Status      Command
-----
[1]    520     Completed   sleep 3
[2]    521     Completed   sleep 5
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```

**TASK 2:** To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Foreground Job Control\n");
    printf("Parent PID: %d\n", getpid());

    // Create a child process
    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("\nChild Process Started\n");
        printf("Child PID: %d\n", getpid());

        // Stop the process temporarily
        printf("Child stopping itself...\n");
        raise(SIGSTOP);

        // Continue after receiving SIGCONT
        printf("Child resumed in foreground\n");

        sleep(2);

        printf("Child process completed\n");
        exit(0);
    }

    // Parent process
    printf("\nParent Process\n");
    printf("Target Job PID: %d\n", pid);

    sleep(1);
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
    perror("fork failed");
    return 1;
}

if (pid == 0) {
    // Child process
    printf("\nChild Process Started\n");
    printf("Child PID: %d\n", getpid());

    // Stop the process temporarily
    printf("Child stopping itself...\n");
    raise(SIGSTOP);

    // Continue after receiving SIGCONT
    printf("Child resumed in foreground\n");

    sleep(2);

    printf("Child process completed\n");
    exit(0);
}

// Parent process
printf("\nParent Process\n");
printf("Target Job PID: %d\n", pid);

sleep(1);

// Resume the stopped child process
printf("Resuming target job...\n");
kill(pid, SIGCONT);

// Wait for child completion
printf("Waiting for foreground job to complete...\n");
waitpid(pid, &status, 0);

if (WIFEXITED(status)) {
    printf("Job state: Completed\n");
    printf("Exit status: %d\n", WEXITSTATUS(status));
}

printf("Foreground switching test completed\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session17_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session17_task2.c -o session17_task2
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session17_task2
Foreground Job Control
Parent PID: 915

Parent Process
Target Job PID: 916

Child Process Started
Child PID: 916
Child stopping itself...
Resuming target job...
Waiting for foreground job to complete...
Child resumed in foreground
Child process completed
Job state: Completed
Exit status: 0
Foreground switching test completed
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```

## Session – 18

**TASK 1:** To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
keerthi_priya@LAPTOP-OSGV  GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <stdlib.h>

void handle_signal(int sig) {
    printf("\nSignal %d received by process %d\n", sig, getpid());
}

int main() {
    sigset_t mask;

    printf("Signal Handling and Process Groups\n");
    printf("Process ID : %d\n", getpid());
    printf("Process Group ID : %d\n", getpgrp());

    // Install signal handlers
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    // Create a signal mask
    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);

    printf("\nBlocking SIGINT for 5 seconds...\n");

    // Block SIGINT
    sigprocmask(SIG_BLOCK, &mask, NULL);

    printf("SIGINT is currently blocked.\n");
    printf("Try pressing Ctrl+C now.\n");

    sleep(5);

    // Unblock SIGINT
    printf("\nUnblocking SIGINT...\n");
    sigprocmask(SIG_UNBLOCK, &mask, NULL);

    printf("SIGINT is now unblocked.\n");
    printf("Press Ctrl+C to test signal delivery.\n");
    printf("Press Ctrl+\ or send SIGTERM to test another signal.\n");

    sleep(10);

    printf("\nSignal test completed.\n");

    return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session18_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session18_task1.c -o session18_task1
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session18_task1
Signal Handling and Process Groups
Process ID : 1060
Process Group ID : 1060

Blocking SIGINT for 5 seconds...
SIGINT is currently blocked.
Try pressing Ctrl+C now.

Unblocking SIGINT...
SIGINT is now unblocked.
Press Ctrl+C to test signal delivery.
Press Ctrl+\ or send SIGTERM to test another signal.

Signal test completed.
keerthi_priya@LAPTOP-OSGVCE7A:~$ ^C
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```

**TASK 2:** To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verifr Execution, Test Background Recovery

```
keerthi_priya@LAPTOP-OSGV  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    pid_t pid;
    char command[50];
    char status[20];
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;

/* Add job to job table */
void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    job_table[job_count].job_id = job_count + 1;
    job_table[job_count].pid = pid;

    strcpy(job_table[job_count].command, command);
    strcpy(job_table[job_count].status, "Stopped");

    job_count++;
}

/* Display job table */
void show_jobs()
{
    printf("\nJob Table:\n");
    printf("-----\n");
    printf("Job ID\tPID\tStatus\tCommand\n");
    printf("-----\n");
}
```

```
keerthi_priya@LAPTOP-OSGV  GNU nano 8.7.1

    if (job_count == 0)
    {
        printf("No jobs available.\n");
    }

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d]\t%d\t%-10s\t%s\n",
            job_table[i].job_id,
            job_table[i].pid,
            job_table[i].status,
            job_table[i].command);
    }

    printf("-----\n");
}

/* Resume a stopped job */
void resume_job(int job_id)
{
    for (int i = 0; i < job_count; i++)
    {
        if (job_table[i].job_id == job_id)
        {
            if (strcmp(job_table[i].status, "Stopped") == 0)
            {
                if (kill(job_table[i].pid, SIGCONT) == -1)
                {
                    perror("SIGCONT failed");
                    return;
                }

                strcpy(job_table[i].status, "Running");

                printf("Job [%d] resumed successfully.\n",
                    job_id);

                printf("SIGCONT sent to PID %d.\n",
                    job_table[i].pid);

                return;
            }
            else
            {
                printf("Job [%d] is not stopped.\n",

```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
        printf("Job [%d] is not stopped.\n",
               job_id);
        return;
    }
}

printf("Job [%d] not found.\n", job_id);
}

/* Monitor job status */
void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;

        pid_t result = waitpid(
            job_table[i].pid,
            &status,
            WNOHANG | WUNTRACED | WCONTINUED
        );

        if (result == job_table[i].pid)
        {
            if (WIFSTOPPED(status))
            {
                strcpy(job_table[i].status, "Stopped");
            }
            else if (WIFCONTINUED(status))
            {
                strcpy(job_table[i].status, "Running");
            }
            else if (WIFEXITED(status) ||
                     WIFSIGNALED(status))
            {
                strcpy(job_table[i].status, "Completed");
            }
        }
    }
}

int main()
{
    char input[100];
    |
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

int main()
{
    char input[100];

    printf("Session 16 - Stopped Job Recovery\n");
    printf("Commands: run, stop, resume, jobs, update, exit\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        /* Create a stopped background job */
        if (strcmp(input, "run") == 0)
        {
            pid_t pid = fork();

            if (pid < 0)
            {
                perror("fork failed");
                continue;
            }

            if (pid == 0)
            {
                printf("\nChild process started.\n");
                printf("PID: %d\n", getpid());

                /* Stop the child */
                raise(SIGSTOP);

                printf("Child resumed successfully.\n");

                sleep(5);

                printf("Child process completed.\n");
                exit(0);
            }
            else
            {
                add_job(pid, "background task");

                printf("Job [%d] created.\n",
                       job_count);

                printf("PID: %d\n", pid);
            }
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
printf("Job [%d] created.\n",
      job_count);

printf("PID: %d\n", pid);
printf("Job initially stopped.\n");
}

/* Stop a running job */
else if (strcmp(input, "stop") == 0)
{
    if (job_count > 0)
    {
        if (kill(job_table[0].pid, SIGSTOP) == -1)
        {
            perror("SIGSTOP failed");
        }
        else
        {
            strcpy(job_table[0].status, "Stopped");
            printf("Job [1] stopped.\n");
        }
    }
    else
    {
        printf("No jobs available.\n");
    }
}

/* Resume first job */
else if (strcmp(input, "resume") == 0)
{
    resume_job(1);
}

/* Display jobs */
else if (strcmp(input, "jobs") == 0)
{
    update_jobs();
    show_jobs();
}

/* Update states */
else if (strcmp(input, "update") == 0)
{
    update_jobs();

    printf("Job status updated.\n");
    show_jobs();
}
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
{
    resume_job(1);
}

/* Display jobs */
else if (strcmp(input, "jobs") == 0)
{
    update_jobs();
    show_jobs();
}

/* Update states */
else if (strcmp(input, "update") == 0)
{
    update_jobs();

    printf("Job status updated.\n");
    show_jobs();
}

/* Exit */
else if (strcmp(input, "exit") == 0)
{
    printf("\nCleaning up jobs...\n");
    for (int i = 0; i < job_count; i++)
    {
        kill(job_table[i].pid, SIGTERM);
        waitpid(job_table[i].pid, NULL, 0);
    }

    printf("Background recovery test completed.\n");
    printf("Exiting shell...\n");

    break;
}

else if (strlen(input) == 0)
{
    continue;
}

else
{
    printf("Invalid command.\n");
    printf("Use: run, stop, resume, jobs, update, exit\n");
}

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session18_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session18_task2.c -o session18_task2
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session18_task2
Session 16 - Stopped Job Recovery
Commands: run, stop, resume, jobs, update, exit
```

```
myshell> run
Job [1] created.
PID: 1251
Job initially stopped.
myshell>
Child process started.
PID: 1251
resume
Job [1] resumed successfully.
SIGCONT sent to PID 1251.
myshell> Child resumed successfully.
jobs
```

Job Table:

| Job ID | PID  | Status  | Command         |
|--------|------|---------|-----------------|
| [1]    | 1251 | Running | background task |

```
myshell> Child process completed.
update
Job status updated.
```

Job Table:

| Job ID | PID  | Status    | Command         |
|--------|------|-----------|-----------------|
| [1]    | 1251 | Completed | background task |

```
myshell> stop
SIGSTOP failed: No such process
myshell> exit
```

```
Cleaning up jobs...
Background recovery test completed.
Exiting shell...
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```